

MLOps Quality & Tracking

Unit Testing · ML Code Validation · Experiment Tracking · Model Registry

A practical reference guide

1. Unit Testing

1.1 Why Unit Testing Matters in ML Projects

ML codebases contain a large amount of conventional software — data loaders, feature transformation functions, pre/post-processing logic, API handlers — alongside the model itself. All of that conventional code benefits from the same unit-testing discipline as any other software: catching regressions early, documenting expected behavior, and enabling safe refactoring.

Unit tests in ML projects are not a substitute for evaluating model quality — they verify that the code around the model behaves correctly, independent of whether the model itself is accurate.

1.2 What to Unit Test in an ML Codebase

- Data transformation and cleaning functions — e.g., does a normalization function produce the expected output for known input?
- Feature engineering logic — e.g., does a 'days since last purchase' feature compute correctly for edge cases (no prior purchase, same-day purchase)?
- Preprocessing/postprocessing pipelines — tokenization, encoding, label mapping, output formatting
- Utility and helper functions — config parsing, path resolution, metric calculation functions
- Model wrapper/interface classes — does the serving wrapper call the model correctly and return the expected shape/type?
- API and pipeline glue code — request validation, error handling, response formatting

1.3 The Testing Pyramid, Adapted for ML

- Unit tests (many, fast) — individual functions: transformations, feature logic, utilities
- Integration tests (fewer) — the full pipeline runs end-to-end on a small sample dataset without errors
- Model/behavioral tests (fewer still) — covered in Section 2; test the trained model's behavior, not just the code
- System/end-to-end tests (fewest) — a deployed serving endpoint returns correct responses for known requests

1.4 Example: pytest for a Feature Engineering Function

```
# features.py
def days_since_last_purchase(reference_date, last_purchase_date):
    if last_purchase_date is None:
        return -1 # sentinel for 'no prior purchase'
    return (reference_date - last_purchase_date).days

# test_features.py
import pytest
from datetime import date
from features import days_since_last_purchase
```

```

def test_days_since_last_purchase_normal_case():
    ref = date(2026, 1, 10)
    last = date(2026, 1, 1)
    assert days_since_last_purchase(ref, last) == 9

def test_days_since_last_purchase_no_prior_purchase():
    ref = date(2026, 1, 10)
    assert days_since_last_purchase(ref, None) == -1

def test_days_since_last_purchase_same_day():
    ref = date(2026, 1, 10)
    assert days_since_last_purchase(ref, ref) == 0

```

1.5 Fixtures and Mocking

- Fixtures (pytest.fixture) provide reusable test data — e.g., a small sample DataFrame used across many tests
- Mocking replaces expensive or external dependencies (a live database call, a cloud storage read, a model inference call) with a lightweight stand-in, keeping unit tests fast and deterministic

```

import pytest
from unittest.mock import patch

@pytest.fixture
def sample_dataframe():
    import pandas as pd
    return pd.DataFrame({'amount': [10, 20, 30], 'label': [0, 1, 0]})

def test_pipeline_runs_on_sample(sample_dataframe):
    result = run_preprocessing(sample_dataframe)
    assert 'amount_scaled' in result.columns

@patch('model_client.predict')
def test_serving_handler_calls_model_once(mock_predict):
    mock_predict.return_value = [0.87]
    response = handle_request({'amount': 25})
    mock_predict.assert_called_once()
    assert response['score'] == 0.87

```

1.6 Test Coverage

Code coverage tools (pytest-cov, coverage.py) report what percentage of lines/branches are exercised by tests. Coverage is a useful signal, not a target to game — 100% coverage with weak assertions is less valuable than 70% coverage with meaningful edge-case tests. In CI, a common pattern is to fail the build if coverage drops below an agreed threshold for critical modules (data and feature code especially).

2. ML Code Validation

2.1 Beyond Unit Tests: Why ML Needs Its Own Validation Layer

Unit tests confirm code behaves as written. ML validation confirms the data and the trained model behave as intended — a different, ML-specific concern that has no equivalent in traditional software testing. A pipeline can pass every unit test and still produce a broken model if the underlying data has silently changed.

2.2 Data Validation

Data validation checks that incoming data matches expectations before it reaches training or inference:

- Schema validation — correct columns, correct types, required fields present
- Range/statistical validation — values fall within expected ranges; distributions haven't shifted dramatically from a known baseline
- Completeness checks — acceptable levels of missing values per field
- Referential/consistency checks — categorical values belong to an expected set; foreign-key-style relationships hold

Common tools

- Great Expectations — declarative data quality 'expectations' with automated validation reports
- TensorFlow Data Validation (TFDV) — schema inference and drift/skew detection between training and serving data
- Pandera / Pydantic — lightweight, code-based schema validation for DataFrames and structured inputs

```
# Example: schema validation with Pandera
import pandera as pa

schema = pa.DataFrameSchema({
    'age': pa.Column(int, checks=pa.Check.in_range(0, 120)),
    'income': pa.Column(float, checks=pa.Check.ge(0)),
    'region': pa.Column(str, checks=pa.Check.isin(['north', 'south', 'east',
    'west'])),
})

validated_df = schema.validate(raw_df) # raises SchemaError on violation
```

2.3 Model / Behavioral Validation

Behavioral tests check the trained model's outputs against expected patterns, independent of overall accuracy. This concept is often organized into three categories (adapted from Jeremy Jordan's ML testing framework and the Checklist methodology for NLP):

- Minimum functionality tests — simple, hand-crafted cases where the correct answer is obvious (a fraud model should flag an unmistakably fraudulent transaction)
- Invariance tests — a change that should NOT affect the prediction doesn't (e.g., changing a customer's name shouldn't change a credit risk score)

- Directional expectation tests — a change that SHOULD move the prediction in a known direction does so (e.g., increasing income should not increase predicted default risk, all else equal)

```
def test_invariance_name_change_does_not_affect_score(trained_model):
    base_input = {'income': 50000, 'age': 30, 'name': 'Alex'}
    changed_input = {**base_input, 'name': 'Priya'}
    assert trained_model.predict(base_input) ==
trained_model.predict(changed_input)

def test_directional_higher_income_lowers_default_risk(trained_model):
    low_income = {'income': 20000, 'age': 30}
    high_income = {'income': 120000, 'age': 30}
    assert trained_model.predict_proba(high_income) <
trained_model.predict_proba(low_income)
```

2.4 Pre-Training vs. Post-Training Validation

Stage	What is validated	Example checks
Pre-training	Data quality before a training run starts	Schema matches, no unexpected nulls, class balance within tolerance, no target leakage in features
Post-training	Trained model behavior before deployment	Accuracy/metric above baseline threshold, behavioral tests pass, fairness/slice-based metrics acceptable, inference latency within budget

2.5 CI Integration

Both data validation and model behavioral tests are typically wired into the CI/CD pipeline as automated gates: a training pipeline that fails data validation should not proceed to training, and a newly trained model that fails behavioral or performance-threshold tests should not be promoted to the model registry's Production stage.

3. Experiment Tracking

3.1 What is Experiment Tracking?

Experiment tracking is the systematic logging of every training run's inputs (data version, code version, hyperparameters, environment) and outputs (metrics, artifacts, trained model) so that results are comparable, reproducible, and auditable over time.

3.2 What to Track for Every Run

- Parameters — hyperparameters, model type, random seed, feature set version
- Metrics — training/validation/test performance, logged per epoch or step where relevant
- Artifacts — the trained model file, plots (confusion matrix, ROC curve), sample predictions
- Code version — the exact Git commit hash used for the run
- Data version — which dataset snapshot or DVC/data-lake version was used
- Environment — library versions, hardware used (useful when results are hardware-sensitive)

3.3 Common Tools

Tool	Notes
MLflow	Open-source; combines experiment tracking, a model registry, and project packaging; self-hostable or managed (Databricks)
Weights & Biases (W&B)	Rich visualization and collaboration features; popular for deep learning research teams
Neptune.ai	Lightweight, flexible metadata store; strong for comparing many runs at scale
Comet	Similar feature set to W&B; strong CI/CD and model-monitoring integrations
SageMaker Experiments / Vertex AI Experiments	Native tracking inside AWS/GCP managed ML platforms

3.4 Example: Tracking a Run with MLflow

```
import mlflow
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score

mlflow.set_experiment('claims-fraud-detection')

with mlflow.start_run(run_name='rf-baseline'):
    params = {'n_estimators': 200, 'max_depth': 12, 'random_state': 42}
    mlflow.log_params(params)
```

```
model = RandomForestClassifier(**params).fit(X_train, y_train)
preds = model.predict(X_val)

mlflow.log_metric('accuracy', accuracy_score(y_val, preds))
mlflow.log_metric('f1_score', f1_score(y_val, preds))

mlflow.sklearn.log_model(model, artifact_path='model')
mlflow.log_artifact('confusion_matrix.png')
```

3.5 Comparing Runs

Tracking tools provide a UI (or queryable API) to compare runs side by side — sorting by metric, filtering by parameter range, and visualizing how a metric changed as a hyperparameter varied. This turns experimentation from an ad hoc, spreadsheet-driven process into a searchable history that any team member can audit.

Best practice: *Log the resolved configuration for every run automatically, rather than relying on a human to remember to record it — tracking should happen as a side effect of running the code, not a manual extra step.*

4. Model Registry

4.1 What is a Model Registry?

A model registry is a centralized store for trained model versions, sitting one level above experiment tracking. Where experiment tracking logs every run (including failed or discarded ones), the model registry holds only models deemed worth keeping — with a clear lifecycle stage, approval history, and lineage back to the run and data that produced them.

4.2 What a Registry Typically Stores

- The serialized model artifact itself (see Section 4 of the previous document for serialization formats)
- Version number, automatically incremented with each new registration
- Lifecycle stage — e.g., None, Staging, Production, Archived
- Lineage — link back to the experiment run, code commit, and data version that produced the model
- Approval/audit metadata — who promoted the model, when, and why (often with required sign-off in regulated environments)

4.3 Model Lifecycle Stages

Stage	Meaning
None	Registered but not yet reviewed or assigned a stage
Staging	Passed initial validation; under further testing (e.g., shadow deployment, A/B test)
Production	Approved and actively serving live traffic
Archived	Superseded by a newer version or retired; kept for audit/rollback purposes

4.4 Common Tools

- MLflow Model Registry — open-source, integrated with MLflow's experiment tracking
- Amazon SageMaker Model Registry — native AWS integration with approval workflows and CI/CD triggers
- Azure ML Model Registry — integrated with Azure ML workspaces and endpoints
- Vertex AI Model Registry — native GCP integration with Vertex AI Endpoints

4.5 Example: Registering and Promoting a Model with MLflow

```
import mlflow
from mlflow import MlflowClient

# Register the model logged in a run to the registry
```

```

result = mlflow.register_model(
    model_uri='runs:/<run_id>/model',
    name='claims-fraud-detector'
)

client = MlflowClient()

# Promote the newly registered version to Staging after passing validation
client.transition_model_version_stage(
    name='claims-fraud-detector',
    version=result.version,
    stage='Staging'
)

# After further review, promote to Production
client.transition_model_version_stage(
    name='claims-fraud-detector',
    version=result.version,
    stage='Production',
    archive_existing_versions=True # auto-archive the prior Production model
)

```

4.6 Registry-Driven CI/CD

In a mature MLOps setup, promotion between stages is itself a controlled, auditable event — often triggered automatically once post-training validation (Section 2.4) passes, or gated behind a manual approval step for Production promotion in regulated contexts. A transition to Production can itself trigger a deployment pipeline that rolls the new model version out to a serving endpoint, closing the loop between the registry and live infrastructure.

Key idea: *Experiment tracking answers ‘what did we try, and what happened?’ A model registry answers ‘which version is actually running in production right now, and how did it get there?’ — they are complementary, not interchangeable.*

Summary

Together, these four practices form the quality and governance backbone of MLOps: unit tests protect the conventional code around a model, ML-specific validation protects against silent data and model failures, experiment tracking makes every training run reproducible and comparable, and a model registry provides a single, auditable source of truth for what is actually running in production.